

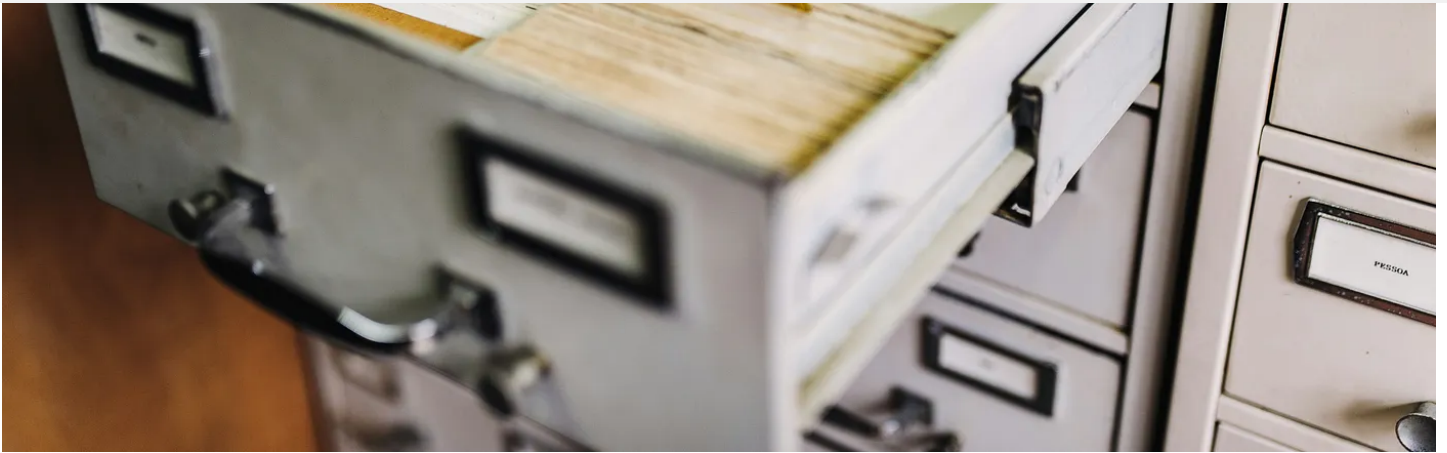


Photo by [Maksym Kaharlytskyi](#) on [Unsplash](#)

Stop losing your best AI conversations: archive to Obsidian with a skill



Bulent Gorkem

Follow

7 min read · May 6, 2026



35



A practical tutorial: build a personal corpus of AI conversations in Obsidian for future reference, then graduate from direct MCP calls to a reusable Claude Skill.

TL;DR

- AI conversations are full of decisions, working code, and tradeoffs that vanish the moment you close the tab. Archive the useful ones into Obsidian as structured markdown notes and you build a personal, searchable corpus you can lean on for months.
- Start manual: use the Obsidian MCP server and ask Claude to summarise + save the conversation in a consistent format. Once the format stabilises, promote the recipe into a Claude **Skill** so the workflow runs the same way every time without re-explaining it.
- Skill-authoring shortcuts: the `description` field is your real interface — write it like prompt engineering. Put dependencies in `compatibility` frontmatter (not the body). Include `## Examples` and `## Troubleshooting`. Stay under 500 lines. And don't self-evaluate triggering inside the conversation that wrote the skill — test it cold.

Working example (with code) lives at github.com/bgorkem/bgorkem-skills.

You probably have a Claude (or ChatGPT, or Cursor) tab open right now where something useful is happening. A debugging breakthrough. A design decision with three good options weighed against each other. A working bash one-liner you spent forty minutes refining.

Tomorrow that tab is gone. Or it's still open but buried under nine others. Either way, the *insight* — the reasoning, the tradeoffs, the one snippet that actually worked — evaporates.

This post walks through a workflow I've been using to fix that:

1. Archive the useful AI conversations into my Obsidian vault as durable, searchable notes.
2. Start by doing it manually through the Obsidian MCP server.
3. Once the pattern stabilises, promote it into a Claude **Skill** so it happens automatically the next time I ask.

The second half is a mini-tutorial on Skill authoring — the learnings transfer to any workflow you'd like Claude to handle the same way every time.

Why archive AI conversations at all

A few reasons the habit pays off:

- **Decisions outlive sessions.** “We chose X over Y because Z” is gold six months later when you’re wondering why the code looks the way it does.
- **Working code is precious.** The bash one-liner that finally worked, the SQL that returned what you wanted — these are easier to find in your own notes than in chat history.
- **Cross-conversation linking.** Obsidian’s `[[backlinks]]` let today's debugging session reference last month's architecture call. AI tools can't do that across sessions; your vault can.
- **Search you control.** Your vault is offline, plain markdown, and indexable by anything (Obsidian search, ripgrep, Dataview, whatever).
- **Future you trains on it.** Pasting an old archived note back into a fresh conversation is the cheapest possible “memory” for an AI assistant.

Good archive notes share a shape: a short summary up top, then “Key decisions,” “Code that worked,” “Follow-ups,” and “Related links.” The shape matters because it makes the notes scannable later — you don’t want a wall of chat transcript, you want a reference.

Step 1: Do it manually with the Obsidian MCP

The simplest setup uses the Obsidian MCP server (any of the well-maintained ones work). Once it's connected, Claude can list your vault, read files, append, patch, and create new notes.

The manual workflow looks like this. At the end of a useful conversation, you say something like:

Summarise this conversation and save it to my Obsidian vault under AI/Conversations/ as a markdown file named YYYY-MM-DD - <slug>.md . Use frontmatter with date and tags. Sections: Summary, Key decisions, Code that worked, Follow-ups, Related.

Claude reads the conversation, distills it, and calls the MCP server's "create file" tool to write the note. Done.

This works. I did it this way for a while. But two friction points showed up quickly:

The format drifts. One day I'd get a note with `## Summary`, the next day `## TL;DR`, the next day no summary at all. The instructions were in my head, not anywhere durable.

I had to re-explain the workflow every time. Every new conversation, fresh context. "Summarise this... use frontmatter... AI/Conversations folder... not the whole transcript, just decisions and code..." It got old fast.

When you find yourself copy-pasting the same instructions into every session, that's the signal. It's time to graduate from a manual prompt to a Skill.

Step 2: What is a Skill, and why use one here

A Skill is a folder with a `SKILL.md` file inside it. The frontmatter contains a `name` and a `description`; the body contains instructions, examples, and troubleshooting notes. Claude loads the Skill automatically when your request matches its description.

In other words: you write the "summarise and save" instructions *once*, in `SKILL.md`. From then on, when you say "save this conversation" or "archive this for later," Claude finds the skill, loads the instructions, and does the right thing — same format every time, no re-explaining.

This is the real upgrade from “direct MCP usage” to “Skill-wrapped workflow.” The MCP server still does the file I/O. The Skill is the *recipe* that tells Claude how to use it consistently.

Step 3: Authoring the Skill

I packaged my version as `obsidian-archive` and put it on GitHub at github.com/bgorkem/bgorkem-skills. Here's what I learned writing it, with notes from Anthropic's [Complete Guide to Building Skills for Claude](#).

The frontmatter is the real interface

Claude looks at the `description` field to decide whether to load your skill at all. Treat it like prompt engineering — it's the only thing visible at the matching stage.

yaml

```
---
name: obsidian-archive
description: Distils the current conversation into a concise reference note and
license: MIT
compatibility:
  mcp-servers:
    - mcp-obsidian
metadata:
  author: bgorkem
  version: 1.1.0
---
```

A weak description = an under-triggered skill. Mention the trigger phrases your users will actually say (“archive this,” “save this conversation,” “log it for later”).

Anatomy of a SKILL.md — what each part is for, and why it matters.

Dependencies belong in `compatibility`, not in the body

This was my biggest first-draft miss. I had a `## Dependencies` heading listing the MCP server I needed. The guide's canonical home for "what MCP servers, plugins, or system packages does this skill require" is the `compatibility` frontmatter field. Move it there.

Add ## Examples and ## Troubleshooting

Examples teach Claude *when* to fire the skill — show a couple of triggering phrasings and what the expected output looks like. Troubleshooting catches the predictable failures: missing MCP server, wrong vault path, archive folder doesn't exist yet, file already exists at the target path.

These two sections are what separate “works on my machine” from “works for anyone who installs it.”

Stay under the 500-line progressive-disclosure target

The Skills system is built around the idea that the `SKILL.md` is the index, and longer reference material lives in a `references/` folder Claude only reads when needed. Mine landed at 181 lines. If yours is creeping past 500, that's the signal to extract.

You can't self-evaluate the skill in the conversation that wrote it

The conversation that authored the skill has the whole context primed; *of course* it triggers the skill correctly. To know whether your description actually triggers in the wild, run a separate test pass: 10–20 varied phrasings (“save this,” “log it,” “write this up,” “add to my notes,” “remember this for later”). Either manually in Claude.ai, or with `run_loop.py` from Claude Code. Don't skip this.

Step 4: Sharing it

Once the skill works, putting it on GitHub costs almost nothing and makes it easy to install across machines. A few small choices that paid off:

Author-scoped repo name. I almost called the repo `devx-skills`. I picked `bgorkem-skills` instead. The repo's scope will drift over time — Obsidian today, maybe NestJS scaffolds or n8n templates next. An author-scoped name survives scope changes; a topic-scoped name forces a rename or a split later.

Mirror the `anthropics/skills` convention. A repo-level `README.md` for human visitors (separate from each skill's `SKILL.md`), one skill per top-level folder, MIT `LICENSE`, `.gitignore`.

```
bgorkem-skills/  
├── .gitignore  
├── LICENSE  
├── README.md  
└── obsidian-archive/  
    └── SKILL.md
```

Push to github

Add a `claude-skills` topic on GitHub for discoverability and you're done.

The pattern, generalised

The arc here works for almost any AI workflow you do more than twice:

1. **Do it manually a few times** through whatever MCP servers you have connected. Notice what's working and what's drifting.
2. **Stabilise the format.** Decide on the exact structure, sections, naming convention.
3. **Wrap it in a Skill.** Move the instructions out of your head and into a `SKILL.md`. Use the frontmatter to make it discoverable, the body to make it deterministic.
4. **Test the trigger** with varied phrasings.
5. **Publish it** so future-you (and anyone else) can install it once and forget about it.

`obsidian-archive` is the first skill in my repo. The second one is going to be one I'm already doing manually too often — that's the only criterion that matters.

If you build one, I'd love to see it.

[Ai Skills](#)[Obsidian](#)[Claude](#)[AI](#)[Ai Archive](#)



Written by Bulent Gorkem

27 followers · 45 following

Follow

Human as a Programmer. AI enthusiast. Explorer of new languages, frameworks, algorithms in computing.
